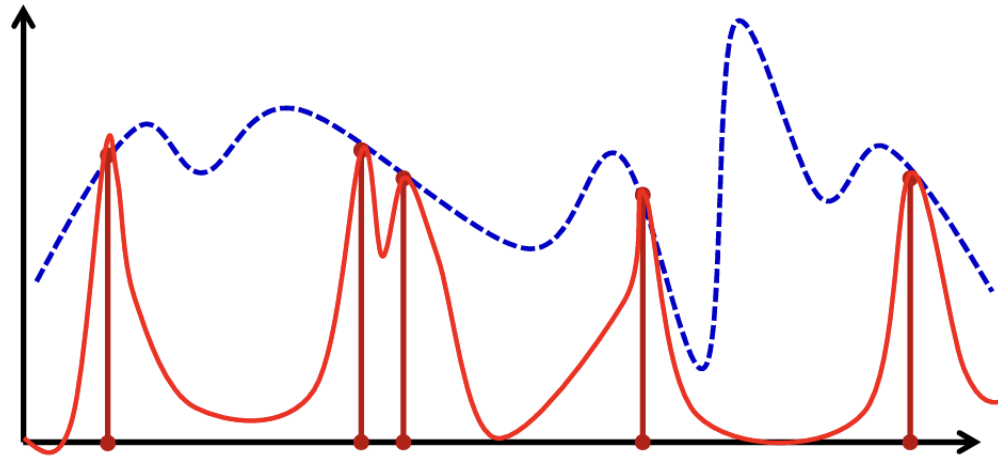


Lecture 8 Model Selection and Generative Models I

Recap: Overfitting



- Problem: ML algorithms may just learn the values of the inputs if the number of parameters in a model is **a lot more than** the true function
- Need additional “**smoothing**” constraints that will “fill in” the missing regions acceptably
- Generalization

Smoothness Through Weight Constraints: L2 Regularization

- Regularized loss function:

$$\ell(f_\theta) = \underbrace{\frac{1}{n} \sum_{i=1}^n \text{Div} \left(y^{(i)}, f_\theta(x^{(i)}) \right)}_{\text{Learning Objective}} + \underbrace{\frac{1}{2} \lambda \|\theta\|_2^2}_{\text{L2 penalty}}$$

- $\|\theta\|_2^2 = \sum_{j=1}^d \theta_j^2$
- λ : tunable regularization parameter

L1 Regularization (Lasso)

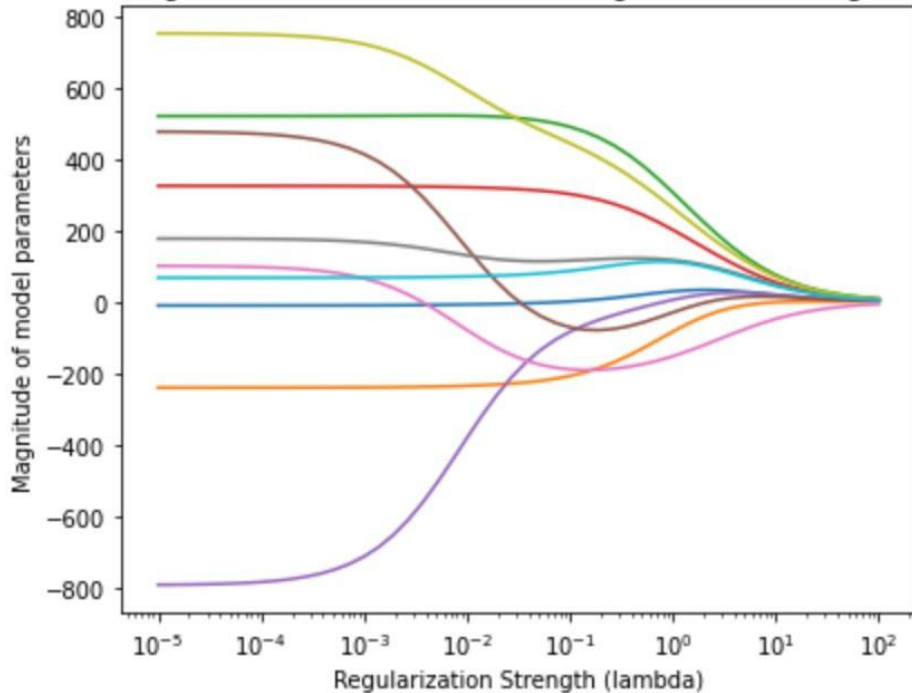
- L2 penalty term: $\frac{1}{2} \lambda \|\theta\|_2^2$
- L1 regularization (L1 norm of θ):

$$\ell(\theta) = \frac{1}{n} \sum_{i=1}^n \text{Div}(y^{(i)}, f_{\theta}(x^{(i)})) + \lambda \cdot \|\theta\|_1$$

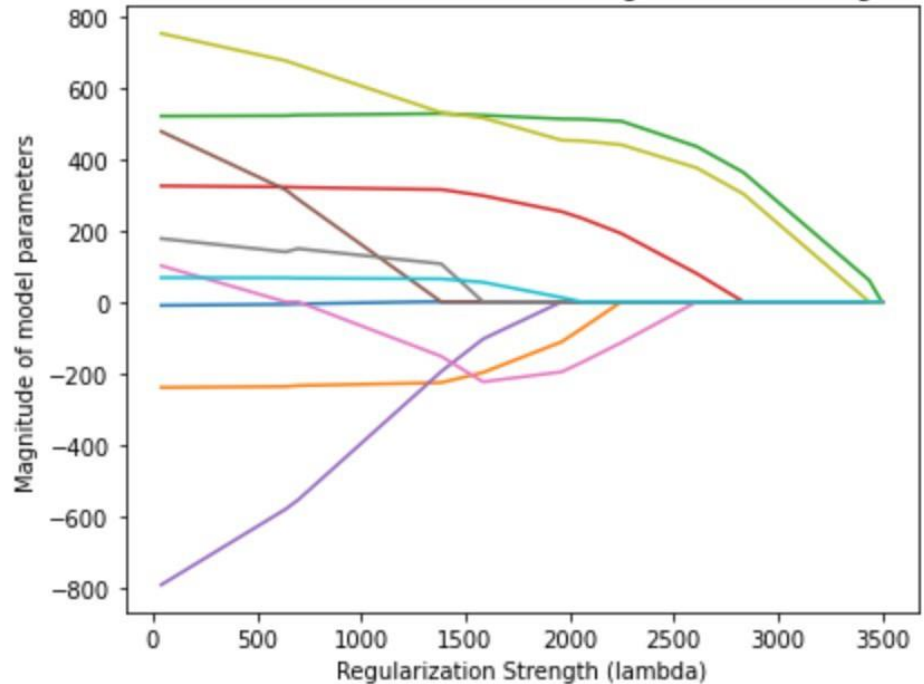
- $\|\theta\|_1 = \sum_{j=1}^d |\theta_j|$
- Forces weights to decay to zero rather than being small
- Optimizer: subgradient descent, coordinate descent, least angle regression, and etc.

- Lasso parameters become progressively smaller
- Stay at zero when the parameters reach 0

Ridge coefficients as a function of regularization strength λ



LASSO coefficients as a function of regularization strength λ



Regularizing via Constraints: example

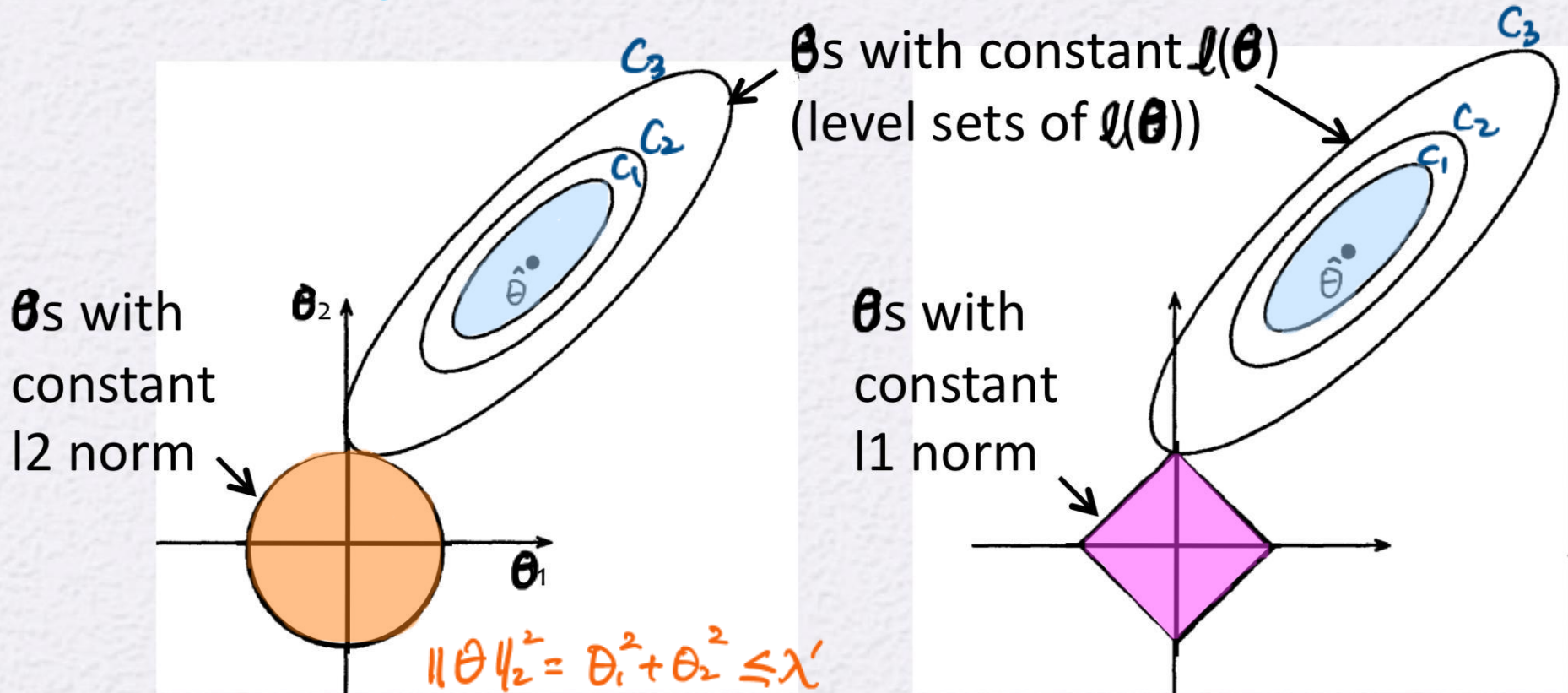
We can view regularization as a constrained optimization problem:

$$\min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n \text{Div}(y^{(i)}, f_{\theta}(x^{(i)}))$$

such that $\|\theta\| \leq \lambda'$

- $\|\cdot\|$: either L1 or L2 norm

$$\frac{1}{n} \sum_{i=1}^n D_{\text{iv}}(y^{(i)}, f_{\theta}(x^{(i)})) = C_{\tilde{\theta}} \text{ for some constant } C_{\tilde{\theta}}$$



$$\|\theta\|_1 = |\theta_1| + |\theta_2| \leq \lambda'$$

optimization objective: find the smallest $C_{\tilde{\theta}}$ such that $\bigcirc$ intersects $\bigcirc$ or $\blacklozenge$

Are the following statements on regularization correct?



L1 and L2 regularizations are general methods that can be used to control the value of the model parameters to prevent overfitting. They work beyond linear regression.

0

L2 induces sparse model parameters.

0

Instead of regularization, when we detect the current model overfits, it is always better to change to a model with fewer model parameters instead.

0

Identifying the model class with the precise number of parameters required to represent a given dataset is straightforward

0

Takeaways

- When training dataset is small, models can overfit
- Can be detected using the held-out development or test set
 - Sampled from the same data distribution
- Overfitting can be mitigated by regularization
 - L2 or L1 norm of θ of the objective

Datasets for Model Development

- When developing ML models, it is customary to work with:
 - Training set: data on which we train our algorithms.
 - Development set (validation or holdout set): data used for hyperparameter tuning.
 - Test set: data used to evaluate the final performance of the model

Model Development Workflow

- Training: try a new model and fit it on the training set
- Model Selection:
 - Estimate performance on the development set using metrics.
 - Based on results, try a new model idea in step 1
- Evaluation: estimate real-world performance on test set

Validation and Test Sets

How to choose the development and test set?

- Distributional consistency:
 - The development and test sets should be from the data distribution that we will see in production
- Dataset size:
 - Should be large enough to estimate future performance
 - 30% of the data on small tasks
 - Usually up to not more than a few thousand instances

Cross Validation

- 16 data points in total
 - 4 data points are reserved for testing
- If we keep 4 data points for validation -> only left with 8 data points for training
- Too small
- Q: how to estimate validation error without a validation dataset?

Cross Validation

Q: how to estimate validation error without a validation dataset?

- Split 12 data points into 3 folds:
 - Each fold contains 4 samples
 - Train the model with a fixed model class and hyperparameter on 2 folds and test on the hold out fold
 - Sample size is small so we might not get a good representation of the data distribution from these points
 - Solution: repeat training by holding out each of the 3 folds and take the average!

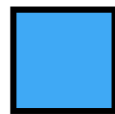
Cross Validation

- K-fold cross-validation: group the data into K disjoint folds.
- Train the model K times, each time using a different fold for testing, and the rest for training
- The training and validation errors of **a fixed model class and fixed hyperparameter set** are estimated by averaging the results from the K experiments

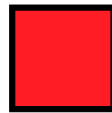
$n = 12$

$k = 3$

Data



Test



Train



Cross Validation

- K-fold cross-validation: group the data into K disjoint folds.
- Train the model K times, each time using a different fold for testing, and the rest for training
- The training and validation errors of **a fixed model class and fixed hyperparameter set** are estimated by averaging the results from the K experiments
- Try different hyperparameters and model classes and repeat the entire K-fold process!

Are the following statements on cross-validation correct?

0

Without considering cross-validation, a validation dataset is especially important for model selection when the dataset is small. This is because it is easier to overfit when the number of data points is low

0

Without the validation dataset and cross-validation, the selected model among many different model classes can easily overfit the dataset

0

It is generally recommended to use a validation dataset when tuning hyperparameters and selecting between different model classes, rather than simply splitting the data into training and testing sets.

0

Are the following statements on cross-validation correct?

0

Cross-validation is an alternative method for model selection and hyperparameter tuning when the data is limited

0

The result of cross-validation proposes a new prediction model

0

When we fix the model class and hyperparameter by using cross-validation, it is a good idea to retrain the model using the combined training and validation datasets to boost model performance

0

Generative Models

A New Problem

- All of the previous models that we have seen considered ML algorithms are learning functions
- They can **operate on**, or **process** a given input data
- They can learn to perform these tasks from the data
- Can ML algorithms also **generate** data?
 - And learn to do so from examples

A New Problem



- From a large collection of images of faces, can an ML algorithm learn to *generate* a new portrait
- Generate samples from the distribution of “face” images
 - How do we even characterize this distribution?

A New Problem



- From a large collection of landscapes, can an ML algorithm learn to *generate* new landscape pictures
- Generate samples from the distribution of “landscape” images
 - How do we even characterize this distribution?

ML Algorithms as Generative Models

- We have seen how machine learning algorithms can perform classification or regression
 - Linear regression, logistic regression, softmax regression
- Next step: ML as generative models
 - Model the distribution of *any* data
 - Such that we can draw samples from it

But First...



- What are generative models?
- How to estimate them?
 - Maximum likelihood estimation

What is a Generative Model?

- A model for the probability distribution of a data x
- E.g., a multinomial, Gaussian etc.
- Computational equivalent: a model that can be used to “generate” data with a distribution similar to given data x
- Typical setting: a box that takes in random seeds and outputs random samples like x



Generative Models

- ML as generative models
- Model the distribution of *any* data
- Such that we can draw samples from it



Generative Model in Unsupervised Learning

- A model for the probability distribution of a data x
 - E.g., a multinomial, Gaussian etc.
- Computational equivalent: a model that can be used to “generate” data with a distribution similar to given data x
- Typical setting: a box that takes in random seeds and outputs random samples like x



What about Generative model in supervised learning?

Probabilistic model

- Classification via probabilistic question:
 - What is the most likely label given the features?
 - $p(y|\mathbf{x})$
 - Classifier output prediction $\hat{y}$ given by:

$$\hat{y} = \arg \max_y p(y|\mathbf{x})$$

Recap: Softmax regression

- For each label class y
 - We learn a separate linear regression
 - Use exponential to make it non-negative
 - Normalize the outcome to be a probability distribution
- k -th entry of the predicted vector
 - $p_{\theta}(y = k|\mathbf{x})$

Discriminative Model

- Softmax regression: a discriminative ML model
 - Directly transfers x into a score for each class y
 - The score can be interpreted as defining a conditional probability $p_{\theta}(y|x)$
- Learn the decision boundary between classes
- Not trying to learn the underlying data-generating distribution

Generative Model

- Define a class-conditional distribution

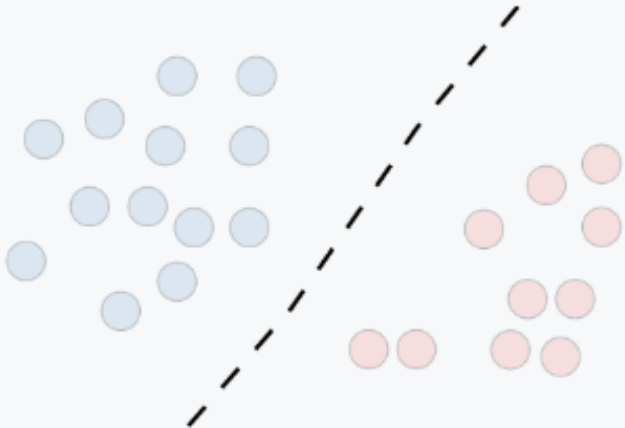
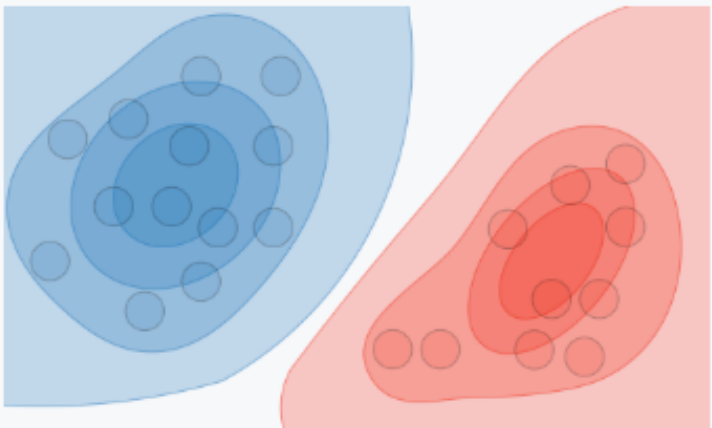
$$P_{\theta}(x|y) : \underbrace{\mathcal{Y}}_{\text{target}} \rightarrow \underbrace{(\mathcal{X} \rightarrow [0, 1])}_{\text{probability } P(x|y) \text{ over } \mathcal{X}}$$

- Parametrization with parameter θ
 - Given a y , write $p_{\theta}(x|y)$ as a function of y and θ
 - The function assigns higher probabilities (“scores”) to more likely x

Generative vs Discriminative Model

- Discriminative, $p_{\theta}(y|\mathbf{x})$:
 - Given a input data $\mathbf{x}$
 - Compute the likelihood of each class y
- Generative, $p_{\theta}(\mathbf{x}|y)$:
 - Given a label class y
 - Compute the likelihood that data point $\mathbf{x}$ is generated from y

Generative vs Discriminative Supervised Learning Model

	Discriminative model	Generative model
Goal	Directly estimate $P(y x)$	Estimate $P(x y)$ to then deduce $P(y x)$
What's learned	Decision boundary	Probability distributions of the data
Illustration	 A scatter plot illustrating a discriminative model. It shows two classes of data points: blue circles on the left and red circles on the right. A dashed black line represents the decision boundary separating the two classes.	 A scatter plot illustrating a generative model. It shows two classes of data points: blue circles on the left and red circles on the right. The data points are enclosed within shaded regions representing probability distributions. The blue region is on the left and the red region is on the right, with some overlap between them.
Examples	Regressions, SVMs	GDA, Naive Bayes

The Advantage of Generative Model

The generative model learns the distribution of the data

- Data imputation and handling missing data
 - Can impute the missing values based on the learned data distribution
- Anomaly Detection
 - Earlier to quantify which points are outliers
- Generate new data samples
 - Generating image to gain interpretability of the model

Are the following statements on generative model correct?



A generative model is a special type of probabilistic model

0

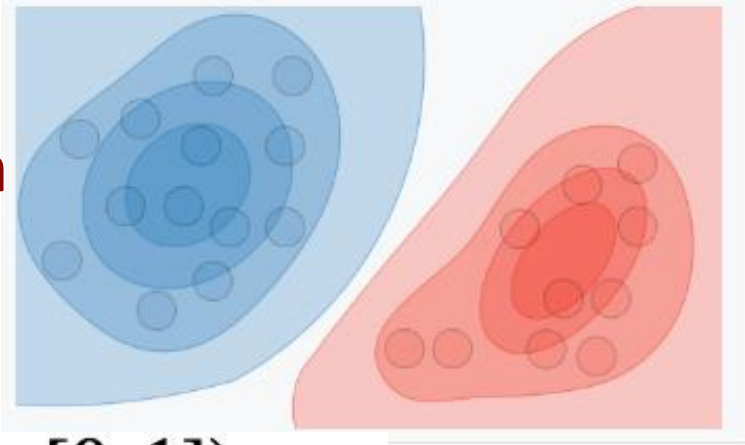
We can fit probabilistic models using maximum likelihood estimation

0

Generative models offer a more complete description of the data than discriminative models by modeling the distribution $P(X|Y)$ instead of $P(Y|X)$

0

Generative Model in Classification



$$P_{\theta}(x|y) : \underbrace{\mathcal{Y}}_{\text{target}} \rightarrow \underbrace{(\mathcal{X} \rightarrow [0, 1])}_{\text{probability } P(x|y) \text{ over } \mathcal{X}}$$

- E.g., spam classification
 - Fit two models, $p_{\theta}(x|y = 0)$, $p_{\theta}(x|y = 1)$
 - $p_{\theta}(x|y = 0)$ scores each x based on whether it looks like non-spam
 - $p_{\theta}(x|y = 1)$ scores each x based on whether it looks like spam

Are the following statements on classification correct?

0

In supervised learning, it is more common to use a generative model for classification than regression.

0

In binary classification, discriminative models learn $p_{\theta}(y|x)$ while generative models learn $p_{\theta}(x|y)$. Furthermore, $p_{\theta}(x|y=0) + p_{\theta}(x|y=1)$ should always equal 1.

0

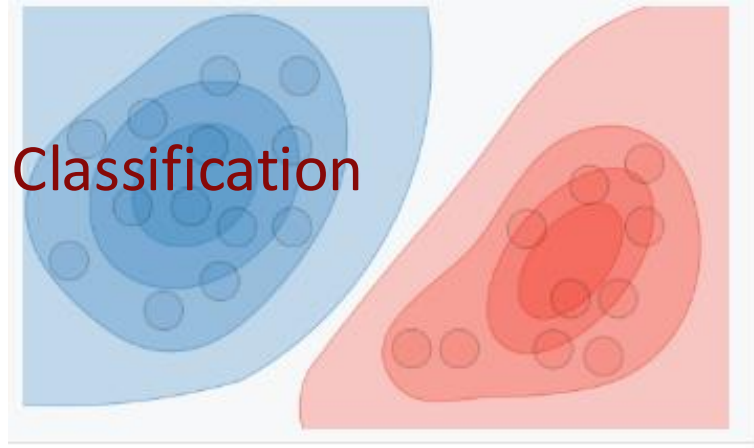
The goal of generative classification model is to eventually learn $p_{\theta}(y|x)$ so that we can predict the label class associated with each sample.

0

Given a generative binary classification algorithm parametrized by θ , if we observe the following for a given feature x $p_{\theta}(x|y=0)=0.6$, $p_{\theta}(x|y=1)=0.4$, we would always predict x to have label $y=0$ since its likelihood is higher.

0

Prediction Using Generative Model in Classification



- Classification:
 - $p_{\theta}(x|y)$ alone is not enough
 - We also need to consider the probability of seeing a class
- Prior probability of seeing $p_{\theta}(y = k)$ of seeing class k

$$P_{\theta}(y) : \underbrace{\mathcal{Y} \rightarrow [0, 1]}_{\text{probability over } \mathcal{Y}}$$

- Spam classification: % of data with class k

Bayes Rule

$$\hat{y} = \arg \max_y p(y|\mathbf{x}) = \arg \max_y \frac{p(\mathbf{x}|y)p(y)}{p(\mathbf{x})}$$

- $p(\mathbf{x})$: independent of label y
 - Not important when comparing across prediction class
- $\hat{y} = \arg \max_y \frac{p(\mathbf{x}|y)p(y)}{p(\mathbf{x})} = \arg \max_y p(\mathbf{x}|y) p(y)$

Prediction on New Data

$$\hat{y} = \arg \max_y p(\mathbf{x}|y)p(y)$$

- Given a new $\mathbf{x}'$
 - Score $\mathbf{x}'$ using each the model $p(\mathbf{x}'|y)$ for each class k
 - Adjust these scores using our priors $p(y = k)$
 - Output the class that assigns the highest score to $\mathbf{x}'$

Generative vs Discriminative Model

- A generative model defines $p_{\theta}(x|y)$ and $p_{\theta}(y)$
- Defines the joint distribution $p_{\theta}(x, y)$

$$P_{\theta}(x, y) : \mathcal{X} \times \mathcal{Y} \rightarrow [0, 1]$$

generative model

$$P_{\theta}(y|x) : \mathcal{X} \rightarrow (\mathcal{Y} \rightarrow [0, 1])$$

discriminative model

- Discriminative models don't define any probability over the x 's
- The parameters of discriminative models are NOT designed to learn that
- Generative models do

Are the following statements on generative model correct?



In a supervised learning problem, a generative model can be viewed as directly parameterizing the joint probability of the data. Thus, we can represent the model using the notation $p_{\theta}(x,y): \mathcal{X} \times \mathcal{Y} \rightarrow [0,1]$ where θ corresponds to model parameters.

0

The generative model will assign higher probabilities (“scores”) to more x,y pairs that are more compatible

0

We can learn the parameters of generative models through maximum likelihood principle

0

Maximum Likelihood Learning

- We can learn a generative model $p_{\theta}(x, y)$ by maximizing the likelihood:

$$\max_{\theta} \frac{1}{n} \sum_{i=1}^n \log P_{\theta}(x^{(i)}, y^{(i)}).$$

- Choose parameters θ such that the model P_{θ} assigns a high probability to each training example $(x^{(i)}, y^{(i)})$ in the dataset D .
- This is probability of observing the dataset $D = \{(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n)\}$ if data were IID draws from P_{θ}